

Aceleração de Algoritmos de Estimação da Frequência Cardíaca Fetal Usando um Processador Embarcado em FPGA

Gustavo Pereira
Universidade de Brasília
Brasília - DF
Email: gugagop4@gmail.com

Resumo—O projeto consiste na implementação de um algoritmo de estimação de frequência cardíaca utilizando a abordagem co-projeto *Hardware-Software*. Inicialmente, foi implementado um modelo em Matlab a partir do qual foi possível constatar que o filtro tem o maior consumo de tempo. Assim, este filtro foi implementado em *hardware* dedicado, que permitiu um ganho de desempenho considerável em comparação à implementação em *software*. Apesar da integração entre os módulos em *hardware* e *software* não ter sido completamente realizada, foi possível constatar que o algoritmo do filtro foi acelerado.

I. INTRODUÇÃO

Dentre os problemas mais comuns de ocorrerem em recém-nascidos está a má formação cardíaca, sendo ela a principal causa de mortalidade relacionada a má formação. Em todo o mundo, 1 a cada 125 recém-nascidos apresenta alguma forma de má formação cardíaca, o que constitui um risco à vida do bebê [2]. De forma a analisar este e outros problemas cardíacos que podem ocorrer ao feto, é feito um monitoramento da frequência cardíaca fetal ou FHR (*Fetal Heart Rate*). Este monitoramento pode ser feito utilizando técnicas de processamento de sinais para a extração do fECG (eletrocardiograma fetal) a partir do aECG (eletrocardiograma abdominal).

Os primeiros estudos relacionados ao eletrocardiograma fetal foram realizados no ano de 1906 pelo cientista M. Cremer. O principal problema apontado na época foi a dificuldade de encontrar uma técnica totalmente eficaz de obtenção dos sinais de FHR. Os primeiros métodos utilizados eram invasivos, onde o eletrodo para análise era inserido na mãe e colocado em contato com a membrana que envolve o feto. Este método entrou em desuso devido à possibilidade de romper a membrana, ocasionando em um nascimento prematuro do feto.

Com o avanço da tecnologia pode-se utilizar métodos de processamento automático de sinais e filtros adaptativos para detecção das ondas do tipo R geradas pelo feto e cancelamento da interferência da atividade cardíaca materna nos eletrodos abdominais [3]. Porém, apesar de todo o avanço alcançado, o máximo que se consegue são aproximações, e os resultados não são totalmente precisos.

O filtro LMS adaptativo é o filtro adaptativo mais simples em questão de implementação computacional, além de ser o que gasta menos recursos. O diagrama de blocos para o filtro pode ser visto na Figura 1.

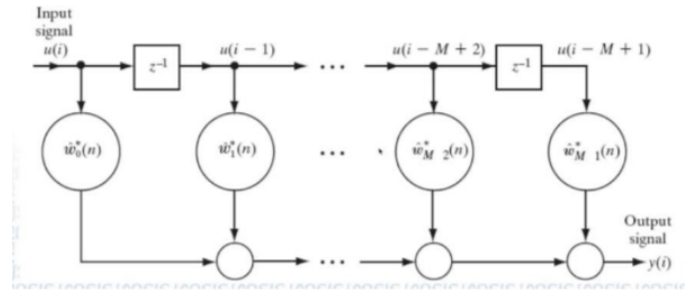


Fig. 1. Diagrama de blocos do filtro LMS

As equações para este modelo de filtro podem ser obtidas abaixo:

$$y[n] = \sum_{k=0}^{M-1} x[n-k] * W_k[n] \quad (1)$$

$$e[n] = d[n] - y[n] \quad (2)$$

$$W_k[n+1] = W_k[n] + \mu u[n-k]e[n] \quad (3)$$

O objetivo do projeto é implementar a aceleração de um algoritmo de estimação que usa um filtro adaptativo dos Mínimos Quadrados. O algoritmo implementado se baseia no mesmo desenvolvido em [1], com a diferença de que o microprocessador escolhido para a implementação neste trabalho é um ARM. Para isso, será utilizada a abordagem co-projeto *Hardware-Software*, demonstrando a diferença de tempo de execução entre a implementação totalmente em software e a implementação que contém *hardware* dedicado para a função de maior custo computacional.

Em resumo, as metas propostas para o projeto são:

- Implementação do software no octave;
- Implementação do software no processador ARM;
- Testes do software;
- Identificação do bloco de maior latência;
- Implementação do bloco em Hardware;

- Teste do co-projeto Hardware-Software;
- Avaliação da melhora de tempo obtida;

II. DESENVOLVIMENTO

Para resolver o problema imposto, será feita a implementação de um filtro adaptativo dos Mínimos Quadrados(LMS) para a filtragem do sinal de ECG da mãe e do feto. Com essa filtragem, teremos o ECG fetal e assim poderemos calcular o FHR. Para esta implementação será utilizado um processador ARM embarcado em uma FPGA, o que atende muito bem o projeto e também garante a possibilidade de implementar novos blocos de maior complexidade no futuro, caso necessário.

Na Figura 2, pode-se observar o diagrama de blocos da arquitetura proposta para o projeto.

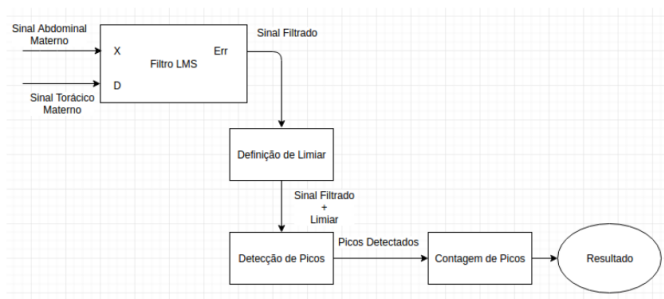


Fig. 2. Diagrama de blocos da arquitetura proposta para o projeto[1]

Nesta arquitetura, temos como sinais de entrada o sinal abdominal e o sinal torácico da mãe. Estes sinais passam por um filtro LMS, onde a saída seria o sinal de ECG fetal filtrado. Com este sinal filtrado, será feito a definição de um limiar, de modo que os picos fiquem acima deste limiar. Desta forma, será possível detectar os picos encontrados e fazer a contagem destes picos, tendo assim o resultado esperado da frequência cardíaca fetal.

Inicialmente, foi necessário gerar sinais de EGC de forma a simular o sinal materno e do feto. Para isso, foi utilizado um algoritmo criado e testado de acordo com as recomendações a Associação dos Avanços em Instrumentação Médica[4].

Para simular um sinal de ECG materno, foi utilizado um número de 89 BPM, gerando um sinal com duração de 10 segundos e com uma taxa de amostragem de 1kHz com uma amplitude máxima de 2 mV. Para simular o sinal de fECG, foi modificado apenas o número de batimentos por minuto para 130 BPM e a amplitude deste sinal foi projetada com uma máxima de 0.2mV.

O resultado da geração destes sinais pode ser obtido na Figura 3.

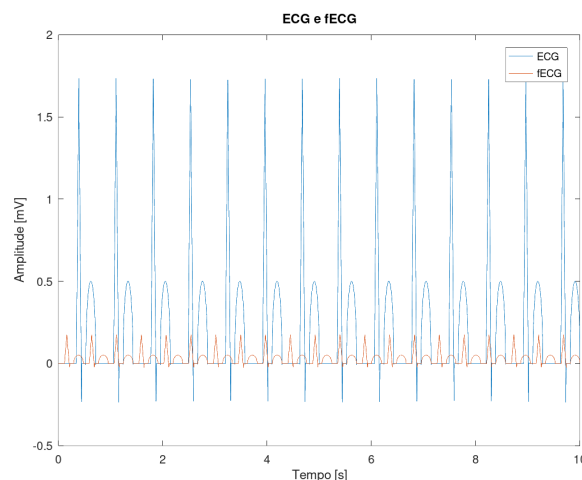


Fig. 3. Sinais simulados de ECG materno e do feto.

Pensando de forma que o sinal de ECG fosse obtido na região abdominal e na região torácica, foi feita a criação destes sinais. Para o sinal abdominal, foi adicionado o ECG materno e um ruído Gaussiano associado. Para o sinal torácico, foi adicionado o ECG materno, o sinal de fECG e também um ruído Gaussiano. O resultado pode ser visto nas Figuras 4 e 5.

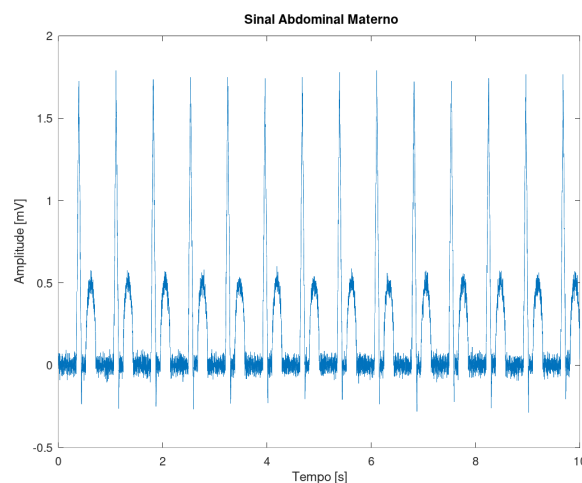


Fig. 4. Sinal abdominal gerado.

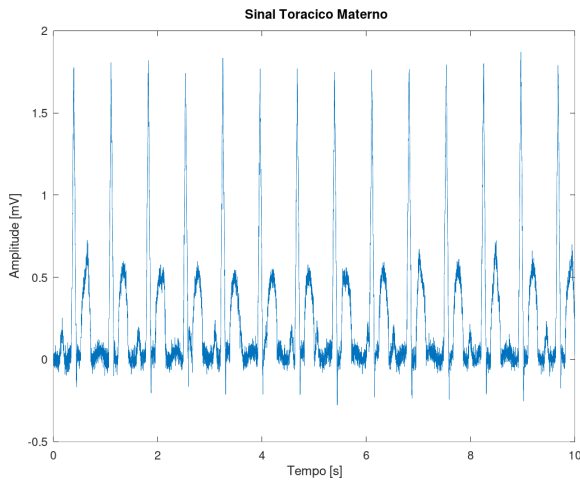


Fig. 5. Sinal torácico gerado.

Após gerados, estes sinais foram armazenados em arquivos em forma de float e binário, cada um com 9440 amostras obtidas representando a captação do sinal ao longo do tempo.

Posteriormente foi criado um modelo de referência no software Octave para o projeto. Este modelo implementa o diagrama de blocos apresentado na Figura 2. Neste modelo, inicialmente foram adquiridos os sinais gerados previamente para serem utilizados.

Ao obter os sinais de teste, eles são inseridos no modelo de filtro adaptativo LMS. Este modelo de filtro obtido tem como entradas o sinal desejado, o sinal de entrada, o parâmetro de tamanho de passo e o número de interações. Como saída temos o sinal de saída, o erro associado e os parâmetros do filtro. Utilizando o sinal desejado como o sinal torácico, o sinal abdominal como sinal de entrada, o tamanho de passo como 0.01 e o número de interações como 20, é obtido um sinal de saída muito próximo ao sinal materno.

A saída do filtro comparado com o sinal materno pode ser visto na Figura 6.

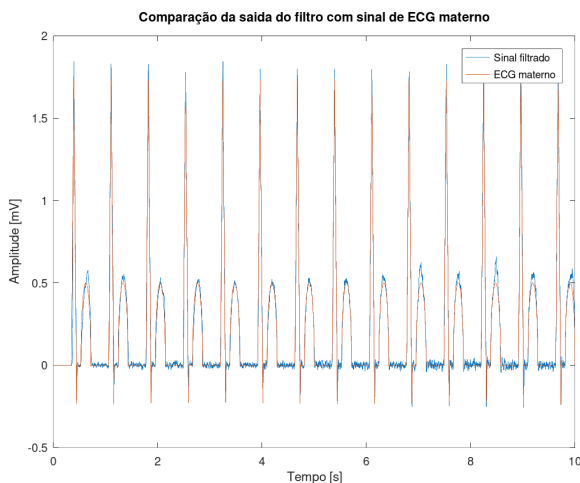


Fig. 6. Comparação da saída do filtro com o ECG materno.

Posteriormente, o bloco de detecção de limiar foi desenvolvido pensando em obter o ponto de maior amplitude e considerar que acima desta amplitude é contado como um batimento. Desta forma, foi obtido o ponto de maior amplitude e traçado um limiar em 70% do valor deste ponto. Assim, foi obtido a linha mostrada na Figura 7.

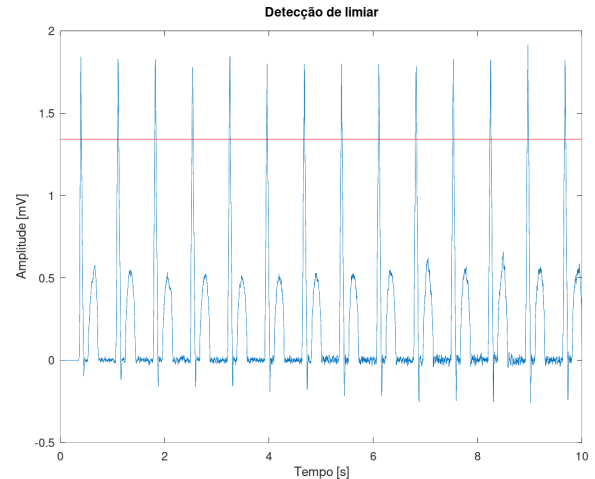


Fig. 7. Linha de limiar detectada.

A partir deste limiar, foram desenvolvidos os blocos de detecção de picos e contagem dos picos, onde a cada vez que a amplitude fosse maior que o limiar selecionado, a número de picos contados aumentaria. Desta forma, foram obtidos 14 picos ao longo de 10 segundos, resultando num sinal com média de 84 BPM. Com isso, foi obtido o número de batimentos materno com erro de 5.61%. Este erro está associado ao baixo tempo do sinal criado.

Com este modelo já implementado na linguagem Matlab, então foi necessário implementar o mesmo na linguagem C, pensando na programação do processador ARM. Com isso, foram implementadas as funções para filtrar o sinal, encontrar o número de picos e calcular o número de batimentos por minuto.

Após feita a implementação em C, foi feito uma estimativa de tempo para realizar o algoritmo em um computador. Para realizar o algoritmo sem o filtro, temos que o tempo de execução é de 7ms para o computador e com o filtro, este tempo de execução vai para 15ms. Desta forma, podemos afirmar que o bloco mais custoso em questão de tempo de execução é o próprio filtro.

Desta forma, este filtro foi implementado em hardware, utilizando as ferramentas do Vivado. O filtro foi implementado pensando em uma posterior filtragem em tempo real, de forma que o número de amostras obtidas é irrelevante para o filtro, levando em conta que os sinais obtidos previamente irão interferir até certo ponto no sinal de saída obtido.

Para a implementação em hardware, foi utilizada a biblioteca de ponto flutuante, aplicando um valor de 1 bit de sinal, 8 bits para a mantissa e 23 bits para o expoente, totalizando 32 bits. Este tamanho de sinal foi pensado para que possam ser utilizados 4 bytes de envio na comunicação entre hardware

e software. O diagrama de blocos para a implementação em hardware pode ser obtido na Figura 8.

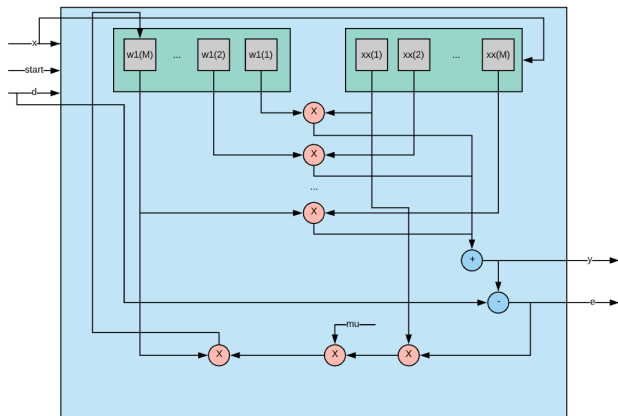


Fig. 8. Diagrama de blocos do hardware do projeto.

Desta forma, o filtro foi implementado em hardware paralelizando multiplicações e somas. Após a implementação, foi gerada uma simulação para testar este filtro. a simulação tinha como base os arquivos de simulação gerados anteriormente. Após a simulação foi possível ver que para as 9440 amostras, esta implementação em hardware estava conseguindo fazer a filtragem em menos de 2.2ms, se mostrando mais rápido que o computador.

Posteriormente, foi gerado um *IP Core* com o filtro criado, de forma que este filtro possa ser implentado utilizando o protocolo AXI lite junto a um processador microblaze. Esta implementação foi feita para que o restante do projeto feito em software possa ser integrado ao hardware.

Feita esta integração, foi feito um teste do co-projeto Hardware-Software, com o intuito de testar a arquitetura criada.

III. RESULTADOS

Na simulação em software utilizando um processador mais potente, o tempo total para o processamento de 9440 amostras foi de aproximadamente 8ms. Esta simulação foi feita utilizando a função time do linux, que calcula o tempo de um programa ser executado.

Após a criação do filtro em hardware, foi obtida uma análise do custo para a sua implementação. A Figura 9 mostra os blocos consumidos por este filtro.

Resource	Estimation	Available	Utilization %
LUT	16623	20800	79.92
FF	5183	41600	12.46
DSP	82	90	91.11
IO	132	106	124.53
BUFG	1	32	3.13

Fig. 9. Tabela de utilização do filtro em hardware.

Também foi feita uma simulação do filtro em hardware. A simulação pode ser vista na Figura 10.

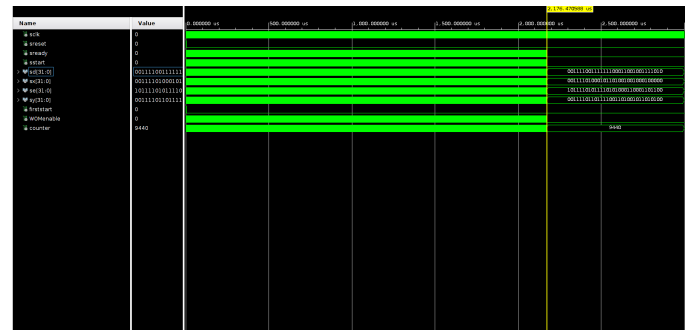


Fig. 10. Simulação do filtro

Na simulação, o tempo total para o processamento de 9440 amostras foi de aproximadamente 2.2ms, dando uma latência de 23 ciclos de clock por amostra.

Ao final da simulação do hardware, foi gerado um arquivo com a saída do filtro, a fim de comparar com a saída do filtro do modelo em Matlab. A Figura 11 mostra a comparação das duas implementações.

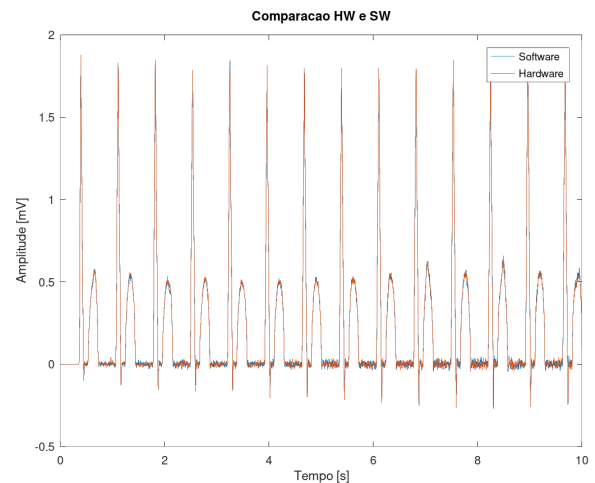


Fig. 11. Comparação da saída do filtro em hardware e software.

Ao compararmos os dois sinais, o MSE obtido foi de 0.0002 e o resultado de saída dos batimentos cardíacos foi igual a implementação em software.

Posteriormente, foi feita uma análise da utilização de recursos do filtro com a utilização do microblaze como processador. A Figura 12 mostra o consumo do hardware da FPGA para esta implementação.

Resource	Utilization	Available	Utilization %
LUT	18433	20800	88.62
LUTRAM	170	9600	1.77
FF	7427	41600	17.85
BRAM	16	50	32.00
DSP	82	90	91.11
IO	4	106	3.77
BUFG	4	32	12.50
MMCM	1	5	20.00

Fig. 12. Tabela de utilização do filtro com o microblaze em hardware.

Na figura 13, podemos encontrar o PAR (*Place and Route*) da implementação. Em vermelho, podemos ver o consumo do filtro com o Axi Lite. Em azul, é visto o restante utilizado para o microblaze.

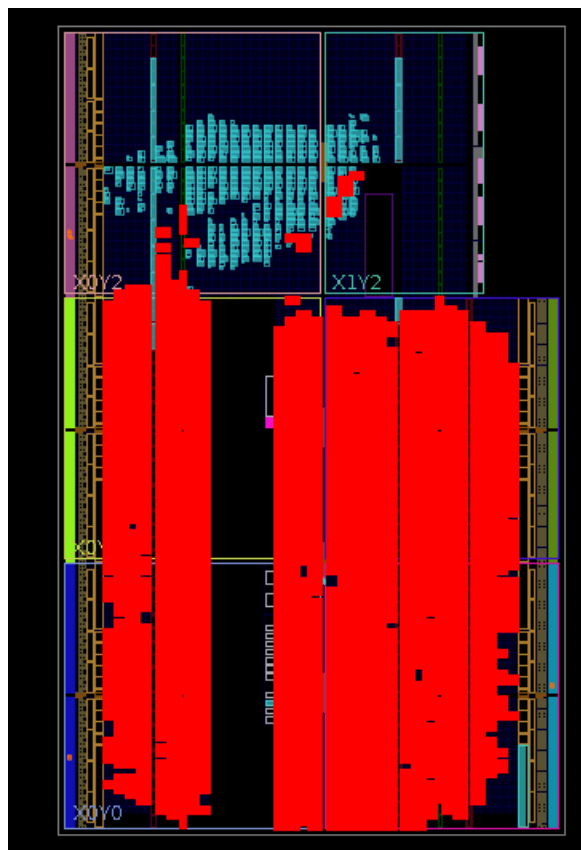


Fig. 13. PAR do filtro com o microblaze.

É possível perceber que o consumo do filtro é muito alto, quando comparado com o consumo do restante do hardware para o projeto. Para o projeto, também foi feita uma análise de consumo energético desta implementação do filtro com o microblaze. O resultado pode ser visto na figura 14.

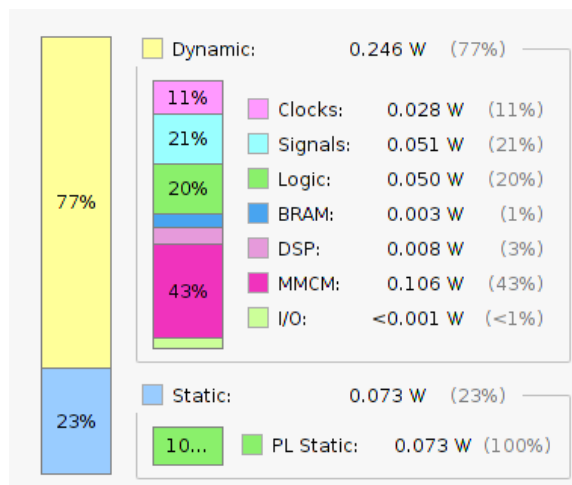


Fig. 14. Consumo energético do filtro com o microblaze.

Ao implementar na FPGA, houve problemas na utilização do projeto na SDK da Xilinx, possivelmente por um problema no software implementado para a comunicação com o protocolo AXI Lite ou devido a um problema de versionamento do programa.

IV. CONCLUSÃO

O projeto tinha como objetivo acelerar um algoritmo de estimação de fECG que utiliza um filtro LMS adaptativo. Durante o projeto, foi analisado que o bloco com maior consumo de tempo era o bloco do próprio filtro. A partir deste dado, foi criado um filtro em hardware que demorasse menos tempo para sua implementação. Ao final do projeto foi possível criar um filtro LMS em hardware que consegue processar os sinais aproximadamente 4 vezes mais rápido que sua implementação em software.

Das metas propostas para o projeto, a única que não foi possível cumprir foi o Teste do co-projeto Hardware-Software, devido a dificuldades encontradas ao longo do projeto.

Como melhorias futuras para o projeto, são sugeridas a diminuição do número de bits da implementação do filtro, com o intuito de reduzir o consumo de blocos na FPGA e poder implementar em placas mais simples. Por fim, é sugerida a utilização de um banco de dados com dados reais, de forma que o filtro possa ser simulado em um ambiente real, visto que sua utilização é pensada para dados obtidos de forma simultânea.

REFERÊNCIAS

- [1] Barbosa, I. J. T. Aceleração de Algoritmos para Estimativa da Frequência Cardíaca Fetal Utilizando FPGA. 2016.
- [2] ASSOCIATION, A. H. Congenital Heart Defects in Children Fact Sheet. 2008.
- [3] WIDROW, B. et al. Adaptive noise cancelling: principles and applications. [S.l.], 1975. v. 63, n. 12.
- [4] Ruha, Antti and Seppo Nissila, "A Real-Time Microprocessor QRS Detector System with a 1-ms Timing Accuracy for the Measurement of Ambulatory HRV", IEEE Trans. Biomed. Eng. Vol. 44, No. 3, 1997
- [5] P. L. Feintuch, "An adaptive recursive LMS filter," in Proceedings of the IEEE, vol. 64, no. 11, pp. 1622-1624, Nov. 1976, doi: 10.1109/PROC.1976.10384.

- [6] K. W. Chan and Y. T. Zhang, "Adaptive reduction of motion artifact from photoplethysmographic recordings using a variable step-size LMS filter," SENSORS, 2002 IEEE, 2002, pp. 1343-1346 vol.2, doi: 10.1109/ICSENS.2002.1037314.
- [7] M. D. Meyer and D. P. Agrawal, "A high sampling rate delayed LMS filter architecture," in IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing, vol. 40, no. 11, pp. 727-729, Nov. 1993, doi: 10.1109/82.251841.